

Applied Mathematics and Nonlinear Sciences

<https://www.sciendo.com>

Development of computer application system and database testing based on data encryption technology

Wei Tang^{1,†}

1. Department of Information engineering, Jiangxi Water Resources Institute, Nanchang, Jiangxi, 330000, China.

Submission Info

Communicated by Z. Sabir

Received February 7, 2023

Accepted July 7, 2023

Available online December 27, 2023

Abstract

In this paper, we use the MapReduce programming framework for the development of a computer application system and design the vision module, control module and transaction module of the system based on MVC architecture. Focusing on the encryption design of the database, the data stored in the database is chunked through the Map function, and all the chunking results are aggregated based on the Reduce function, and synchronized encryption of the database is combined with the Paillier homomorphic encryption algorithm. On this basis, the system environment is configured, and the application system developed in this paper is tested, focusing on exploring the stability of the database. The test results show that the decryption output time is 21s when the Paillier encryption algorithm is 200, and the first set of database peak is 76% of the server load, which is within the tolerable range. Security test on the database; the results show that the probability of an attack on all ports with a database security factor less than 6 is around 0.3. There is no significant difference, and the database test is good.

Keywords: Mapreduce programming; Application development; MVC architecture; Paillier homomorphic encryption; Database.

AMS 2010 codes: 97M50

[†]Corresponding author.

Email address: 3378421838@qq.com

1 Introduction

With the continuous expansion of computer applications, the development of database systems has increased accordingly, and at the same time, the problems exposed in the development process of database systems are also increasing [1-3]. Such as, database systems are designed to be more and more complex, and the need to deal with the data information gradually increases. From time to time, data redundancy occurs [4-5]. This will largely affect the performance of the computer application system developed, as well as the design function [6].

In the development of computer applications, it is necessary to ensure that the time and manpower to meet the needs improve the database testing technology research and development efforts [7-8]. The introduction of scientific and advanced technology and the optimization of the database testing work in order to ensure that the database system achieves normal operation [9]. Only in this way can the performance of computer application system development be optimized and upgraded [10-11].

Data encryption technology is of great importance in information security technology. Literature [12] proposes a LUT method that implements RSA with minimum processing time for encryption of information privacy data and compares it with the standard algorithms such as the Chinese remainder theorem binary and square multiplication methods, and it is found that the method has a better performance than the other standard methods as the size of the exponent value increases. Literature [13] proposed a new VL communication technique based on asymmetric encryption for data encryption, determined the optimal key length for public or private encryption under the minimum estimation error criterion, and carried out practical experiments on the technique using an implemented testbed to explore the encryption effect of the algorithm. Literature [14] In order to improve the performance of data security in the cloud and to provide data processing for different parties, an efficient multi-party secure data access management method is proposed to improve the performance of all the quality of service parameters of the cloud by assigning the key set and scheme set to different users with user-centric characteristics. Literature [15], in order to develop more secure data encryption, incorporates a nature-inspired optimization algorithm to select the correct and optimal elliptic curve values, combines DNA coding with the ECC encryption algorithm and evaluates the performance of this data encryption algorithm based on the encryption time, decryption time, throughput and key size of the security model.

Database testing when an important part of the development of computer applications. Literature [16] uses alternating optimization and differential convex programming optimization methods to optimize the database-assisted dynamic spectrum allocation and proposes a database-assisted dynamic spectrum allocation method with spectral security to deal with more serious security issues. Literature [17] to protect the security of computer networks, proposed a model to automatically update the attack list using a filtering engine as a second IDS engine and automatically update the IDS database with new attack signatures for database security.

In this paper, firstly, we sort out the specific steps to develop the system under the MapReduce framework, and by analyzing the advantages of the model-view-controller pattern architecture, we adopt MVC to make the visual module, control module and transaction module in the system. Secondly, the data encryption algorithm is designed and implemented under the MapReduce system framework, and for the shortcomings of the Paillier algorithm's low running efficiency, parallel addition is adopted to improve the running speed. Finally, the performance of the parallel Paillier homomorphic encryption algorithm is analyzed. Two groups of test cases were created to test the database's stability and security, respectively. The first group of tests employed the database to perform certain tasks, and the database's condition was observed and examined. The second set of

tests uses a randomly generated DNS to access the database and evaluates its reliability based on its operating conditions. Additionally, the security evaluation of the information system was completed by performing system attack tests such as SYNflood.

2 MAPREDUCE Web application system development

The MapReduce development steps can be divided into the following main steps:

The first step is to use a platform-independent modeling language, e.g., to model the application domain at a high level of abstraction. This model has nothing to do with the technology that implements it or the underlying technology, i.e., platform-independent modeling. The aim is to describe the system requirements as accurately as possible from the user's point of view.

The PIM is then translated into one or more PSMs, a translation process that is generally automated. The PSM describes the application system by describing a specific implementation technology. It will utilize the various architectures provided by this technology, such as EJBs, database models, COM components, and so on.

Finally, the PSM will be translated into source code. Since each PSM already relies exclusively on one technology, this step is usually relatively straightforward. When MapReduce reaches this stage, the development of the software system is basically finished, and the user receives the desired software system.

To facilitate developers' development and maintenance, the framework utilizes the model and view separation pattern in this paper. The design pattern uses MVC, i.e. Model-View-Controller pattern. It has a strong advantage in interactive system development, especially in the design pattern for application development.

2.1 Vision module

The main function of the view component is to receive input from the user and display the results of the output system processing to them. In the view component, view controls can be used to form the view, and they are implemented based on the user's manipulation of the platform. By subdividing the common view elements in a web page into reusable view elements, encapsulating these view elements into user controls, and encapsulating the controls into interface controls to be added to the view component system, different interface controls can be used to construct the tail page that the user needs. That is to say, the framework of the platform automatically loads the view controls according to the content of the component model file, inserts these view controls into the view control containers of the web pages, and dynamically generates the various user interfaces of the application platform at runtime. The process is as follows:

When a user accesses a web page, the platform framework sends a message to the control manager and sends a request to the control manager to generate a view control object for the page. When the control manager receives the request from the framework, it also calls the control factory to create various controls that need to be loaded into the page and returns them to the control manager. The Frame framework contacts the Placeholder container and inserts the controls from the control manager into the Placeholder container. After all the view controls of a web page have been loaded and inserted into the corresponding containers, the web page object will automatically complete the drawing of the controls, transform each view control into the corresponding HTML element, form a web page interface, and return to the client browser. Based on the URL of each user visit and the

user's role and privileges, the framework can dynamically load different view controls to form a variety of user interfaces.

2.2 Control module

Define the platform-independent petri net model description and run the definition. The Petri net structure is shown in Figure 1. In the MAPREDUCE-based application development platform, after the XML definition is completed and the system runs the petri net controller engine, the MAPREDUCE platform can interpret the XML definition file into a platform-independent model, thus realizing platform-independent model conversion. An executable Petri net model is achieved by calling this platform-independent model. The XML definition can be modified online without compiling it every time it is activated, which has an impact on the operational state.

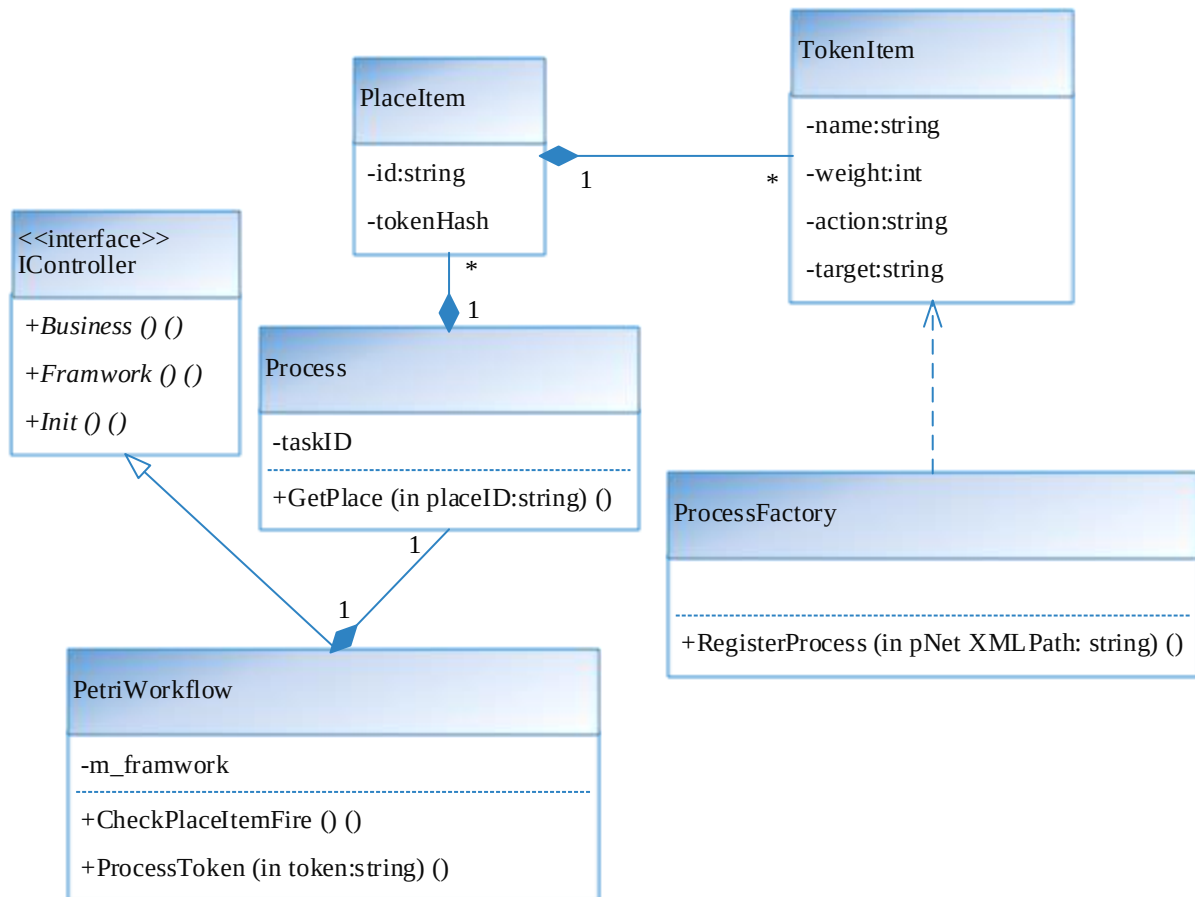


Figure 1. Petri net engine class structure

2.3 Transaction module

The transaction module is composed of transaction components, data connection components and data components to form a complete transaction processing processing framework. This module completes the event processing database connection and other functions. The transaction module's description is depicted in Figure 2. The transaction component, based on the development framework of the event processing mechanism, listens to a specific action. After configuration, transaction processing operations are executed. By refining the reusable transaction module of the application system and encapsulating it into a transaction component, the repeated event processing operations can be greatly reduced, effectively reducing the complexity of coding. At the same time,

the use of transaction components on the application system transactions for the encapsulation of the description can help users to complete the application system transaction modeling work.

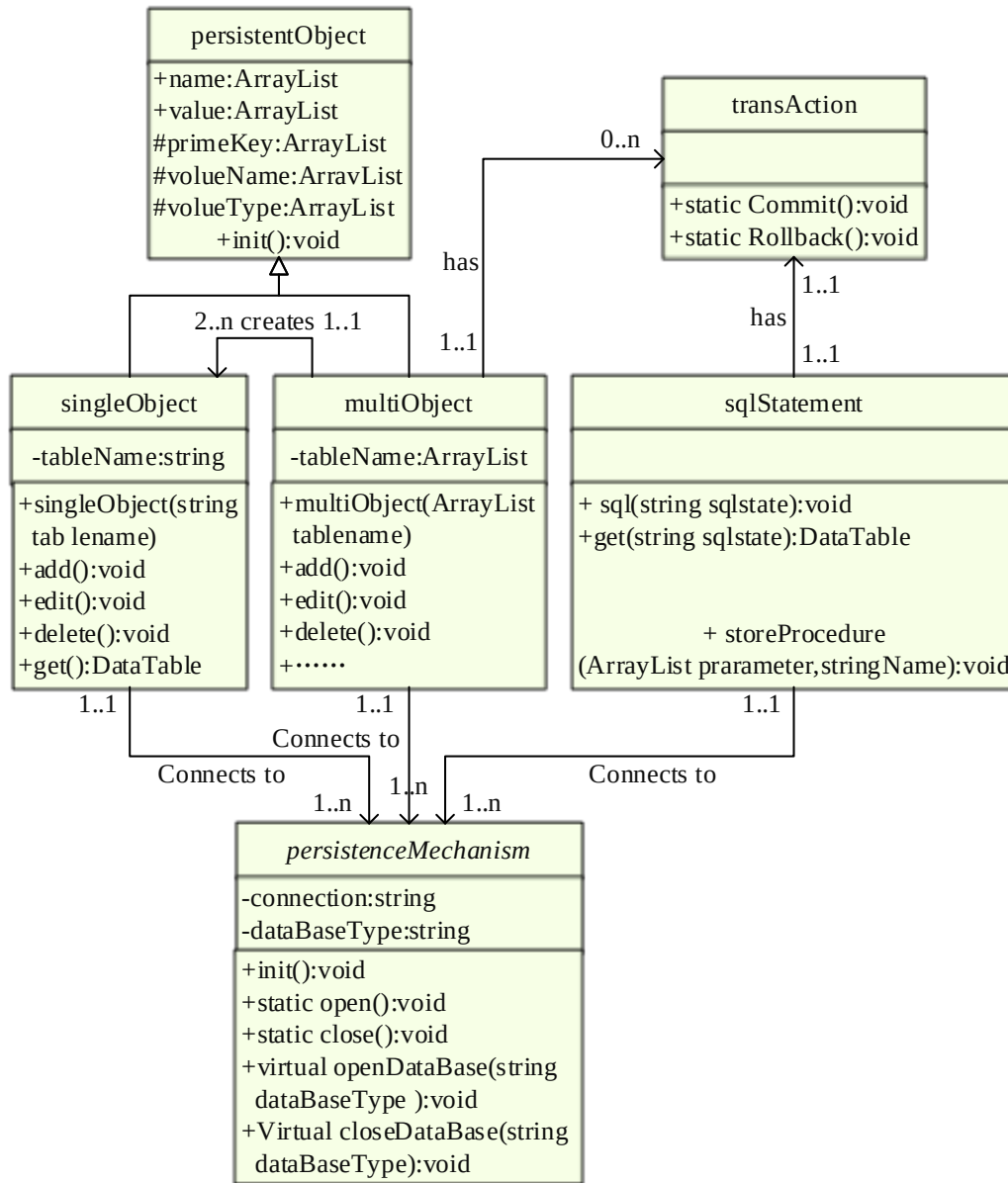


Figure 2. Transaction component design diagram

3 Data encryption algorithms

3.1 Paillier Homomorphic Encryption Algorithm

Paillier encryption algorithm is a probabilistic encryption and the specific encryption process is as follows:

- 1) Key generation: Let $n = pq$, where p, q is 2 large prime numbers, Euler function $\phi(n) = (p-1)(q-1)$. Define set $S_n = \{u | 0 < u < n^2, u = 1 \bmod n\}$, function L as: $L(u) = (u-1)/n$. randomly choose $g \in \square_{n^2}$ such that g satisfies

$\gcd(L(g^\lambda \bmod n^2), n) = 1$, where λ is defined as $\lambda(n) = \text{lcm}(p-1)(q-1)$, public key $kp = \{n, g\}$, private key $sk = \lambda(n)$ or $sk = p, q$.

- 2) Encryption process: have any plaintext $m \in \mathbb{Z}_n$ and choose random number $r \in \mathbb{Z}_n^*$, then the resulting ciphertext is:

$$c = E(m) = g^m r^n \bmod n^2 \quad (1)$$

- 3) Decryption process: using private key λ and ciphertext c , use the decryption formula to obtain plaintext m :

$$m = D(c) = \frac{L(c^\lambda \bmod n^2)}{L(g^\lambda \bmod n^2)} \bmod n \quad (2)$$

Additive homomorphism is proved by assuming that c_1, c_2 is the encrypted ciphertext of plaintext m_1, m_2 , respectively, then $c_1 = g^{m_1} r_1^n \bmod n^2$, $c_2 = g^{m_2} r_2^n \bmod n^2$, then the product of the two ciphertexts c_1, c_2 is:

$$E(m_1) \cdot E(m_2) = c_1 \cdot c_2 = (g^{m_1} r_1^n \bmod n^2) (g^{m_2} r_2^n \bmod n^2) = g^{m_1+m_2} (r_1 r_2)^n \bmod n^2 \quad (3)$$

And the plaintext $m_1 + m_2$ is encrypted using the encryption formula, which can be obtained:

$$E(m_1 + m_2) = g^{m_1+m_2} r^n \bmod n^2 \quad (4)$$

Since r is a random number, $r = r_1 \times r_2$, i.e., $E(m_1 + m_2) = E(m_1) \cdot E(m_2)$, can be taken, so the Paillier encryption algorithm has additive homomorphism.

3.2 Design of Parallel Homomorphic Encryption Scheme for MapReduce

The encryption algorithm in this paper mainly utilizes the additive homomorphism of Paillier's algorithm for encryption, and according to the current running speed of the computer, the length of the public key n is 256 bits, which can ensure the security of the encryption algorithm, and the parallel encryption design scheme proposed in this paper is as follows:

According to the definition of Paillier encryption algorithm, public key $\{n, g\}$ and private key $\lambda(n)$ are generated.

Let the plaintext file of size $S(\text{MB})$ can be divided into X groups of length 256 bits, then the number of groups X is:

$$X = \left\lceil \frac{S \times 2^{20}}{32} \right\rceil \quad (5)$$

For each grouping, first convert the plaintext to data of type BigInteger with a value of m . Set $m = m_1 + m_2$. Calculate the values of m_1 and m_2 , and if m is even, then $m_1 = m_2 = m / 2 (m \bmod 2 = 0)$, and if m is odd, then order:

$$\begin{cases} m_1 = (m-1) / 2 \\ m_2 = (m+1) / 2 \end{cases} (m \bmod 2 = 1) \quad (6)$$

Calculate $E(m_1)$ and $E(m_2)$ separately using the encryption equations as follows:

$$\begin{cases} E(m_1) = g^{m_1} r_1^n \bmod n^2 \\ E(m_2) = g^{m_2} r_2^n \bmod n^2 \end{cases} \quad (7)$$

Where, g, n is the public key, and g is a random number, $g \in \mathbb{Z}_{n^2}^*$, r_1, r_2 is a random number, and $r_1, r_2 \in \mathbb{Z}_n^*$, using the above equation we can see that the plaintext m is encrypted to generate the ciphertext c as:

$$c = E(m_1) \times E(m_2) \quad (8)$$

If m is an even number, then $m_1 = m_2$, which may be taken as $r_1 = r_2$, is $E(m_1) = E(m_2)$. At this point we only need to calculate $E(m_1)$, which is $c = E(m_1)^2$.

In the decryption process, according to the additive homomorphism of Paillier's algorithm, $E(m) = E(m_1) \times E(m_2) \bmod n^2$ can be calculated, and using the decryption formula, the plaintext $m = L(c^\lambda \bmod n^2) / L(g^\lambda \bmod n^2) \bmod n$ can be calculated, where c is the ciphertext $E(m)$.

The implementation of the parallel Paillier homomorphic encryption algorithm is based on the MapReduce parallel programming framework. In the MapReduce programming model, the Split function splits the input data into fixed-size blocks according to the user's needs, and then the master node. It is assigned to different processors according to the corresponding scheduling mechanism. The Map function performs the corresponding operations on each data block after slicing according to the user-defined algorithm, and each Map runs the partially encrypted data. The Reduce function aggregates the results produced by all the Maps. The encryption computation of each data block of the Paillier parallel homomorphic encryption scheme is carried out independently, and therefore it can be distributed to multiple Map to perform encryption simultaneously.

The Split function is used to process the input task, and the input parameter key-value pair (K, V) represents each data slice. When initializing the MapReduce task, the size of each slice needs to be calculated in order to evenly distribute the sliced data to each core. Let the original file be divided into t data blocks ($t \geq 1$), i denotes the i th data block, $Blocksize_i$ denotes the size of the i th slice ($i \geq n$), the key length is 256 bits, and the size of each plaintext block is an integer multiple of 32 bytes, and the specific byte size of each slice of $Blocksize_i$ can be expressed by Equation (9).

$$Blocksize_i = \begin{cases} \left\lceil \frac{S \times 2^{20}}{(32 \times t)} \right\rceil \times 32, i < t \\ S \times 2^{20} - (t-1) \left\lceil \frac{S \times 2^{20}}{(32 \times t)} \right\rceil \times 32, i = t \end{cases} \quad (9)$$

3.3 Algorithm Performance Analysis Methods

The evaluation of parallel computing can be roughly divided into machine-level performance evaluation, algorithm-level performance evaluation and program-level performance evaluation, and the algorithm-level performance evaluation mainly includes the acceleration ratio, efficiency and scalability, etc. The main disadvantage of the Paillie encryption algorithm is the relatively slower encryption speed, and the main purpose of researching parallel Paillier homomorphic encryption algorithms is to improve the encryption speed of the algorithms, so in the performance analysis, we mainly consider three factors: the Map speedup ratio, the overall speedup ratio of the parallel algorithm and the efficiency.

The serial overall encryption time of Paillie's algorithm is the key generation time T_{seq} , then T_{seq} can be expressed by the formula: $T_{seq} = T_{key} + T_{Enc}$. According to the encryption formula, it can be seen that the main operation in the Paillier encryption algorithm is the modulus operation, assuming that a modulus to be operated on the time overhead is T_{pd} , then the time overhead to carry out an encryption operation is $2T_{pd}$, according to the previous setting of the key length of 256 bits, the file size is $S(MB)$ of the plaintext is divided into x groups, then the time required to encrypt the plaintext $T_{Enc} = 2 \times x \times T_{pd}$, serial encryption of the plaintext file requires a time of $T_{seq} = T_{key} + 2 \times x \times T_{pd}$, according to the above formula can be obtained:

$$T_{seq} = T_{key} + 2 \times \left\lceil \frac{S \times 2^{20}}{32} \right\rceil \times T_{pd} = T_{key} + S \times 2^{16} \times T_{pd} \quad (10)$$

When the value of S is large, T_{key} is negligible, and it can be obtained that the serial encryption time T_{seq} basically tends to increase linearly as the file size S increases.

In the parallel encryption process, assuming that there is no overlap in the operation process, the execution time T_n of the parallel encryption algorithm consists of four parts, namely:

$$T_n = T_{comm} + T_{key} + T_{Map} + T_{reduce} \quad (11)$$

Where, T_{comm} is the communication time, T_{Map} is the time for Map parallel encryption, and T_{reduce} is the time for merging the encrypted ciphertexts according to the key value.

Assuming that the T_i total number of CPU cores is p , the plaintext with file size $S(MB)$ is divided into t pieces, and the processing time of a Map is T_i , then:

$$T_n = T_{comm} + T_{key} + \max(T_i) \times \left\lceil \frac{t}{p} \right\rceil + T_{reduce} \quad (12)$$

Based on the size of each block $Blocksize$ given in the above equation, it can be deduced that each block of plaintext data contains the number of packets $x_i = \left\lceil \frac{S \times 2^{20}}{32 \times t} \right\rceil$, then:

$$T_i = 2 \times \left\lceil \frac{S \times 2^{20}}{32 \times t} \right\rceil \times T_{pd} \quad (13)$$

Let S_T be the speedup ratio of Map in the parallel encryption phase:

$$S_T = \frac{T_{Enc}}{T_{comm} + T_i} = \frac{S \times 2^{16} \times T_{pd}}{T_{comm} + 2 \times \left\lceil \frac{S \times 2^{20}}{32 \times t} \right\rceil \times T_{pd}} \quad (14)$$

When the amount of data in the file S has a large value and T_{comm} is negligible, then:

$$S_T = \frac{T_{Enc}}{T_{comm} + T_i} \approx \frac{S \times 2^{16} \times T_{pd}}{2 \times \left\lceil \frac{S \times 2^{20}}{32 \times t} \right\rceil \times T_{pd}} \approx \frac{S}{\left\lceil \frac{S}{t} \right\rceil} \quad (15)$$

It can be seen that when the value of file size S is an integer multiple of the number of chunks t , the Map acceleration ratio can reach the maximum theoretically t . For plaintext files of the same size, the sizes of the generated ciphertexts are also basically the same, and assuming that the Reduce time needed to summarize the ciphertexts of each subgroup is T_{rci} , the Reduce time required for a plaintext with a file size of $S(MB)$ is:

$$T_{reduce} = x \times T_{rci} = \left\lceil \frac{S \times 2^{20}}{32} \right\rceil \times T_{rci} \quad (16)$$

It can be shown that T_{reduce} is proportional to the size of the generated ciphertext and can be obtained at the same time:

$$T_n = T_{comm} + T_{key} + 2 \times \left\lceil \frac{S \times 2^{20}}{32} \right\rceil \times T_{pd} \times \left\lceil \frac{t}{p} \right\rceil + \left\lceil \frac{S \times 2^{20}}{32} \right\rceil \times T_{rci} \quad (17)$$

The overall acceleration ratio S_p can be expressed as:

$$S_p = \frac{T_{seq}}{T_n} = \frac{T_{key} + T_{Enc}}{T_{comm} + T_{key} + T_{Map} + T_{reduce}} \quad (18)$$

When the file size is large, T_{comm} and T_{key} can be ignored, then:

$$S_p \approx \frac{2 \times \left\lceil \frac{S \times 2^{20}}{32} \right\rceil \times T_{pd}}{2 \times \left\lceil \frac{S \times 2^{20}}{32 \times t} \right\rceil \times T_{pd} \times \left\lceil \frac{t}{p} \right\rceil + \left\lceil \frac{S \times 2^{20}}{32} \right\rceil \times T_{rci}} \approx \frac{1}{\frac{1}{t} \times \left\lceil \frac{t}{p} \right\rceil + \frac{1}{2} \times \frac{T_{rci}}{T_{pd}}} \quad (19)$$

It can be seen that when the total number of cores p is fixed, $t \in (kp, kp + p]$, the acceleration ratio S_p tends to increase, and when the number of chunks t is an integer multiple of the total number of cores p , the acceleration ratio S_p reaches its maximum value and the maximum does not exceed the total number of processor cores p .

Efficiency is the degree to which the parallel system acceleration ratio S_p approaches the number of processors p , which is expressed as:

$$E = \frac{S_p}{p} \approx \frac{1}{\frac{1}{t} \times \left\lceil \frac{t}{p} \right\rceil + \frac{1}{2} \times \frac{T_{rci}}{T_{pd}}} \times \frac{1}{p} = \frac{1}{\frac{p}{t} \times \left\lceil \frac{t}{p} \right\rceil + \frac{p}{2} \times \frac{T_{rci}}{T_{pd}}} \quad (20)$$

Through the formula can be seen, when the total number of nuclei p fixed, $t \in (kp, kp + p]$, with the increase of t , the efficiency E also gradually increase: when $t = kp$, the efficiency reaches the maximum value, but always less than 1. When the number of chunks t fixed, $p < t$, and $t \in [kp, kp + p)$, with the increase of p the efficiency of the decreasing trend; when $t = kp$, the efficiency reaches the maximum value. When $p > t$, the efficiency gradually decreases with the increase of p .

4 Application system testing and analysis

Table 1 shows the test models as well as the environment.

Table 1. Database test environment

Host computer(1)	Master
System	Ulbuntu16.04
Internal storage	2GB
Disk	80GB
Software and information	JDK:openjdk 1.8 Zookeeper:3.4 Hadoop:2.7 HBase:2.0 Phoenix:4.0
Host computer(2)	Slave 1
System	Ulbuntu16.04
Internal storage	2GB
Disk	80G
Software and information	JDK:openjdk 1.8 Zookeeper:3.4 Hadoop:2.7 HBase::2.0 Phoenix:4.0
Host computer(3)	Slave 2
System	Ulbuntu16.04
Internal storage	2GB
Disk	80G
Software and information	JDK:openjdk 1.8 Zookeeper:3.4 Hadoop:2.7 HBase:2.0 Phoenix:4.0

4.1 Algorithm Performance Analysis

This experiment is mainly to test the stand-alone environment Paillier encryption algorithm in the encryption and decryption process takes time, the use of Java language to achieve the encryption and decryption process of the Paillier encryption algorithm, the experimental process of the selection of the key length of 256-bit, so a one-time can be read into the 32-bit characters, due to the value of a larger number of, it is the use of Java's BigInteger BigInteger type in Java is used because of the large value.

The experimental environment is divided into hardware and software environments, with hardware using a computer and specific hardware and software configurations, as shown in the table. The size of the plaintext data selected for the experiment is 200MB, 400MB, 600MB, 800MB, and 1GB. The specific encryption and decryption time obtained after testing is shown in Table 2. It can be seen that as the plaintext data increases, the encryption and decryption time basically increases linearly, and the overall decryption time is slightly longer than the encryption time. Due to the large expansion rate of the ciphertext generated by the Paillier encryption algorithm, although multiple characters are encrypted at a time in this experiment, the expansion rate of the ciphertext still reaches about 8 times, and when the plaintext gradually increases, the generated ciphertext also increases proportionally, which leads to a rapid increase in the time needed to write the ciphertext.

We can also see that the Paillier encryption algorithm takes a long time to encrypt and decrypt and its overall efficiency is low. The algorithm has a low efficiency overall.

Table 2. Encryption software situation

Planitext size(MB)	Encryption time(s)	Decryption(s)	Input-output time(s)	Total time(s)
200	1235	1438	21	2691
400	2530	2951	46	5530
600	3710	4300	68	8170
800	5100	5940	103	11163
1000	6250	7356	129	13750

4.2 Database analysis

4.2.1 Database Case I

For databases, stability needs to focus on query time, as well as metrics related to hardware resources. At the same time, the success rate of the query and other indicators that are more important for traditional transactional databases are not important. Therefore, the relevant indicators include stability of query response time, stability of CPU utilization, stability of disk I/O, stability of memory occupancy and so on. Among them, the query response stability needs to calculate the response time stability of each type, as well as the response time stability of a mixture of various types. In this paper, the stability of response time, the stability of CPU utilization, the stability of disk I/O, and the stability of memory occupancy are selected as the test metrics.

Figure 3 shows the number of tasks completed per unit of time using computational data. The number of bytes reaches the peak when the number of tasks is around 2, exceeding 12,000 byte/ms, but then begins to decline, showing a ridge shape, and the number of bytes reaches 8,000 byte/ms when the number of tasks reaches 18, which is lower than the amount consumed when the tasks are 2.

Test results for the data loading test, in addition to the need to monitor the resources of the server where the database is located, the test loading time is an additional needs to be recorded, so it is necessary to decide the data required for the test based on the method used for the data loading test. The data loading test adopts the BulkLoading method. The main process is to load data from the local to HDFS and then import the data into the database through the MapReduce method. Therefore, the loading speed is calculated mainly through the following three data: the number of bytes of the Map task completed per unit of time, the number of bytes of Reduce completed per unit of time, the CPU time required per unit of bytes, and the CPU time required per unit of bytes. The number of bytes completed by the Map task per unit of time, the number of bytes completed by the Reduce process per unit of time, and the CPU time per unit of bytes.

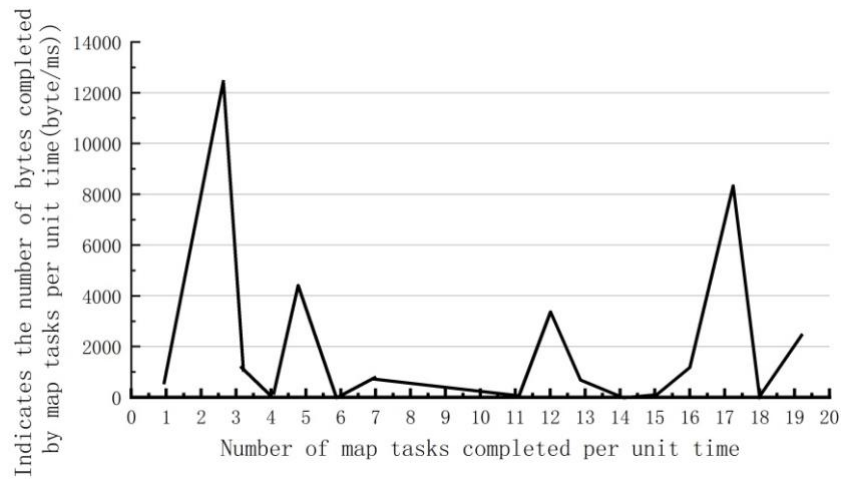


Figure 3. Number of map tasks completed per unit time

Figure 4 displays the memory usage in relation to the number of tasks. The memory utilization increases until there are 7 tasks, but then it begins to show an unstable trend. When the number of tasks reaches 10, memory utilization decreases to about 80%. From the test results, it can be seen that the stability of more complex data mining queries is lower than that of simpler ad-hoc and report queries. At the same time, the number of concurrent users has a less significant impact on the stability of the database.

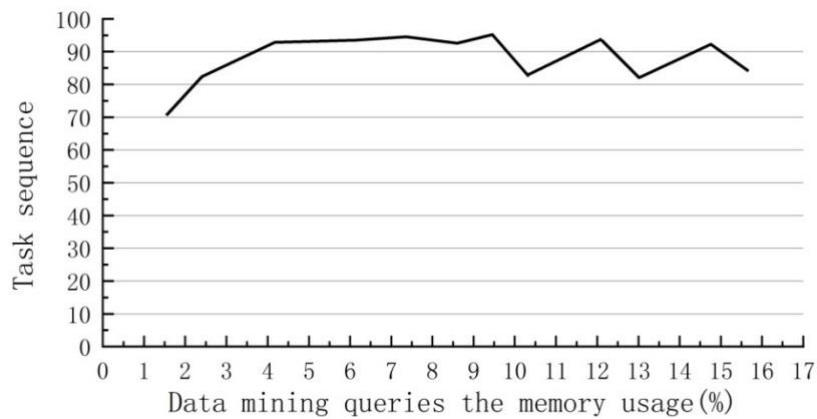


Figure 4. Memory usage

4.2.2 Database case II

MPP database test indicators: For testing functionality, security, and high availability indicators, the testing methods and processes can be summarized in three aspects.

- 1) Generate data phase: for the preparation of data for MPP database functionality, security and high availability testing. The generated data is then entered into the database.
- 2) Generate SQL test statement phase: used for functionality, security, and high availability testing.
- 3) Output results and analysis: It is mainly used to evaluate the time characteristics and resource utilization characteristics of the test SQL statement execution under different data sizes and different nodes.

Table 3 shows the performance test simulation. The simulation is to import the generated DNS table, load the data continuously for 3 hours, and trigger the deletion of historical data once every hour. The DNS table is designed as follows: DDL: CREATE TABLE DNS (SIP INT8, DIP INT8, SPORT INT4, DPORT INT4, CAPTIME INT8, FLAG INT4, TTL INT4, PTC INT2, ISP VARCHAR). In the test, there is no failure or crash, the peak value is maintained at about 74, which is within the affordable range, and the security and functionality of the database are guaranteed. The result shows that the database is stable and can meet the needs of customers in different situations.

Table 3. Performance test situation

Transaction Name	Minimum	Average	Maximum	Std.Deviation	90percent	pass
Actions transaction	0.21	4.082	74.71	6.037	7.79	4.68
drooPrt	1.046	40.6	61.96	28.043	61.96	3
Import1	41.1	53.84	63.59	6.337	62.02	36
data trans02-1	0.26	6.07	7.822	1.126	6.68	37
data trans02-2	0.154	9.5	59.142	11.909	7.72	37
data trans02-3	0.096	6.7	8.86	1.37	8.224	4
data trans3	1.76	3.09	4.203	0.878	4.201	4

4.3 System Security Testing

The deployment of the experimental environment constructed in this chapter includes a management node server, WEB server, file server, data server, firewall, router, attack machine, and legitimate users. The experiment uses SYNflood attack, port scanning and brute-force cracking as the attack methods to establish a large number of incomplete TCP connections with the server so that it cannot respond to the normal service requests from the legitimate users, thus realizing denial of service. The intensity of the attack causes a gradual decrease in the information security quality value of the information system during the attack process. Countering the threat involves deploying security management and control strategies, and the chosen security control strategy involves actively stopping the attack action. The security of the information system is evaluated based on relevant index data obtained from various aspects of system operation. Table 4 displays the security performance test. The ports with high-security importance were not targeted, and their occurrence probabilities of 0.417, 0.270, and 0.333 were not significantly higher than those of the ports with high-security importance, which were maintained at 0.3 or lower. The virus did not recognize the important ports of the database, and the frequency of occurrence did not exceed 0.5, so security is guaranteed to a certain extent.

Table 4. Safety test diagram

Basic security incident	Probability of occurrence	Property	Safety importance
Weak password	0.357	File server	4.243
Channel attack	0.327	Web server	4.243
Unit exception	0.370	Data server	3.647
Identity forgery	0.286	node server	3.634
Access control	0.417	Terminal	6.231
Brute force cracking	0.270	System entry gateway	6.678
Database plaintext transmission	0.333	System entry firewall	6.362

5 Conclusion

In this paper, MapReduce has been used to develop web applications. Based on this, the focus is on encrypting the database. The Paillier homomorphic encryption algorithm was used in encryption, and the following conclusions were obtained:

- 1) In terms of encryption algorithm. This paper proves that Paillier homomorphic encryption algorithm has little impact on the system's operating speed and does not cause a crash or slowdown effect, but the total time when the plaintext size of 200mb is 2,691 seconds, and the total time when the plaintext size of 800mb is already as high as 11,163 seconds. The data encryption method, with the increase of plaintext data, encryption and decryption time basically increases linearly, and the decryption time is slightly longer than the encryption time, which has certain disadvantages.
- 2) In terms of system development and database, MAPREDUCE's Web application system development is more efficient, practical, and has great operability.
- 3) encrypted database operation, this paper on the database for two groups of tests. In the first group of tests, the number of bytes per unit of time Map task completion of 2, the number of bytes per unit of time Map task completion of the number of bytes reached the peak, 12000 then began to decline, to the number of bytes per unit of time Map task completion of 17, the number of bytes per unit of time Map task completion of the number of bytes to reach the top of the peak back to the top. However, no group of bodies crashed or failed to pass the test, which is enough to prove the effectiveness and practicality of this paper's method. In actual evaluation work, the user can be reasonably adjusted according to the actual use of the situation, which is not a problem.

References

- [1] Gobert, M., Nagy, C., Rocha, H., Demeyer, S., & Cleve, A. (2023). Best practices of testing database manipulation code. Information systems.
- [2] Raksawat, C., & Charoenporn, P. (2021). Software testing system development based on iso 29119. Engineering and Technology Publishing(2).
- [3] Villani, EmiliaPontes, Rodrigo PastlCoracini, Guilherme KisseloflAmbrosio, Ana Maria. (2019). Integrating model checking and model based testing for industrial software development. Computers in Industry, 104.
- [4] Martinez-Ledezma, J. A., Barron-Zambrano, J. H., Diaz-Manriquez, A., Elizondo-Leal, J. C., & Rostro-Gonzalez, H. (2020). Versatile implementation of a hardware–software architecture for development and testing of brain–computer interfaces. International Journal of Advanced Robotic Systems, 17(6), 172988142098025.
- [5] Minamino, Y., Inoue, S., & Yamada, S. (2016). Nhpp-based change-point modeling for software reliability assessment and its application to software development management. Annals of Operations Research, 244(1), 1-17.
- [6] Kaliraj, M. R. S., & Bharathi, A. (2018). Path testing based reliability analysis framework of component based software system. Measurement.
- [7] Subramanian, G. H., Pendharkar, P. C., & Pai, D. R. (2017). An examination of determinants of software testing and project management effort. Journal of computer information systems(2).
- [8] Nath, P., Mushahary, J. R., Roy, U., Brahma, M., & Singh, P. K. (2023). Ai and blockchain-based source code vulnerability detection and prevention system for multiparty software development. Computers & Electrical Engineering, 106, 108607-.

- [9] Burnashev, R. A., & Enikeev, A. I. (2016). Structuring of textual data and its processing with database management systems in the development of computer assisted testing systems for students. *International journal of soft computing*, 11(7), 444-447.
- [10] Bukanin, V. A., Ivanov, A. N., & Boriskina, A. V. (2020). Experience of the distance testing system development and operation. *Journal of Physics: Conference Series*, 1691(1), 012170 (6pp).
- [11] Kaydani, H., Mohebbi, A., & Baghaie, A. (2012). Neural fuzzy system development for the prediction of permeability from wireline data based on fuzzy clustering. *Petroleum Science and Technology*(17/20), 30.
- [12] Budati, A. K., Suv, G., Cherukupalli, K., Anil, K. P., & Venkata, K. M. T. (2021). High speed data encryption technique with optimized memory based rsa algorithm for communications. *Circuit World*, ahead-of-print(ahead-of-print).
- [13] Lee, Y. U. (2020). Secure visible light communication technique based on asymmetric data encryption for 6g communication service. *Electronics*, 9.
- [14] Arulananth, T. S., Baskar, M., Anbarasu, V., Thiagarajan, R., Rajendran, T., & Balaji, A. (2021). Multi party secure data access management in cloud using user centric block chain data encryption. *Pattern recognition letters*(152-Dec.).
- [15] Kumar, P., & Bhatt, A. K. (2020). Enhancing multi-tenancy security in the cloud computing using hybrid ecc-based data encryption approach. *IET Communications*, 14(18), 3212-3222.
- [16] Chen, Y., Zhang, Y., Hu, G., & He, P. (2020). Database-assisted dynamic spectrum allocation method with spectrum security. *International Journal of Performability Engineering*(12), 16.
- [17] A, A. Y., & B, N. T. A. D. (2019). Dynamically detecting security threats and updating a signature-based intrusion detection system's database. *Procedia Computer Science*, 159, 1507-1516.